

Q36 – Bellman-Ford vs Floyd-Warshall

Algorithm

1. Aim

To implement the **Bellman-Ford** and **Floyd-Warshall** algorithms for finding shortest paths in a weighted graph and compare their approaches, time complexity, memory usage, scalability, and applications.

2. Problem Statement

Shortest-path algorithms are used to determine the minimum-cost path between vertices in a graph.

Bellman-Ford and Floyd-Warshall solve different versions of the shortest-path problem. Bellman-Ford finds the shortest paths from a **single source vertex** to all other vertices, whereas Floyd-Warshall finds the shortest paths between **all pairs of vertices**.

The objective is to implement both algorithms on the same weighted directed graph and analyze their performance and suitability.

3. Objectives

- Implement Bellman-Ford algorithm.
 - Implement Floyd-Warshall algorithm.
 - Calculate shortest paths using both algorithms.
 - Compare single-source and all-pairs approaches.
 - Analyze time complexity.
 - Analyze memory requirements.
 - Understand handling of negative edge weights.
 - Evaluate scalability for different graph sizes.
 - Identify suitable applications for each algorithm.
-

4. Graph Used

A directed weighted graph containing **5 vertices and 10 edges** is used.

The vertices are:

V0, V1, V2, V3, V4

The graph contains both positive and negative edge weights.

Source	Destination	Weight
V0	V1	6
V0	V2	7
V1	V2	8
V1	V3	5
V1	V4	-4
V2	V3	-3
V2	V4	9
V3	V1	-2
V4	V0	2
V4	V3	7

The source vertex for Bellman-Ford is **V0**.

5. Bellman-Ford Algorithm

Bellman-Ford calculates the shortest distance from one source vertex to every other vertex.

The algorithm repeatedly relaxes all edges for **V - 1** iterations.

It is capable of handling negative edge weights and can detect negative-weight cycles.

Steps

1. Initialize the source distance as 0.
2. Set all other distances to infinity.
3. Relax every edge.

4. Repeat the relaxation process $V - 1$ times.
 5. Perform one additional iteration to detect a negative-weight cycle.
 6. Display the shortest distances.
-

6. Floyd-Warshall Algorithm

Floyd-Warshall calculates the shortest paths between **every pair of vertices**.

It uses a distance matrix and considers each vertex as an intermediate vertex.

Steps

1. Create a distance matrix.
 2. Set diagonal values to zero.
 3. Insert the given edge weights.
 4. Consider each vertex as an intermediate vertex.
 5. Update the distance matrix using the shortest available path.
 6. Check the diagonal for negative values to detect negative cycles.
 7. Display the final shortest-distance matrix.
-

7. Pseudocode – Bellman-Ford

START

Initialize distance[source] = 0

Set all other distances to INF

FOR $i = 1$ to $V-1$

 FOR each edge (u, v, weight)

 IF distance[u] + weight < distance[v]

 distance[v] = distance[u] + weight

 END IF

 END FOR

END FOR

FOR each edge (u, v, weight)

 IF distance[u] + weight < distance[v]

 Negative cycle exists

 END IF

END FOR

Display shortest distances

STOP

8. Pseudocode – Floyd-Warshall

START

Initialize distance matrix

FOR each vertex i

$\text{distance}[i][i] = 0$

END FOR

FOR each edge (u, v, weight)

$\text{distance}[u][v] = \text{weight}$

END FOR

FOR $k = 0$ to $V-1$

 FOR $i = 0$ to $V-1$

 FOR $j = 0$ to $V-1$

$\text{distance}[i][j] =$

$\text{MIN}(\text{distance}[i][j],$

$\text{distance}[i][k] + \text{distance}[k][j])$

 END FOR

 END FOR

END FOR

IF $\text{distance}[i][i] < 0$

 Negative cycle exists

END IF

Display distance matrix

STOP

9. Source Code

The algorithms were implemented using Python.

```

1 # Distance from a Vertex to all other Vertices
2 # Shortest Path Comparison
3
4 INF = float('inf')
5
6
7 # Bellman-Ford Algorithm
8 # Bellman-Ford Algorithm
9
10 def bellman_ford(vertices, edges, source):
11     distance = [INF] * vertices
12     distance[source] = 0
13
14     # Relax all edges V-1 times
15     for _ in range(vertices - 1):
16         updated = False
17         for u, v, weight in edges:
18             if distance[u] != INF and distance[u] + weight < distance[v]:
19                 distance[v] = distance[u] + weight
20                 updated = True
21         if not updated:
22             break
23
24     # Check for negative weight cycle
25     for u, v, weight in edges:
26         if distance[u] != INF and distance[u] + weight < distance[v]:
27             return None, True
28     return distance, False
29
30
31 # Floyd-Warshall Algorithm
32 # Floyd-Warshall Algorithm
33
34 def floyd_warshall(vertices, edges):
35     distance = [[INF] * vertices for _ in range(vertices)]
36
37     # Distance from a vertex to itself is zero
38     for i in range(vertices):
39         distance[i][i] = 0
40
41     # Initialize edge weights
42     for u, v, weight in edges:
43         distance[u][v] = weight
44
45     # Dynamic programming
46     for k in range(vertices):
47         for i in range(vertices):
48             for j in range(vertices):

```

Figure 1: Python implementation of Bellman-Ford and Floyd-Warshall algorithms.

10. Execution

The program was executed using the predefined weighted directed graph.

Bellman-Ford was executed using **V0 as the source vertex**, while Floyd-Warshall calculated shortest paths between every pair of vertices.

```
=====
BELLMAN-FORD VS FLOYD-WARSHALL
SHORTEST PATH ANALYSIS
=====

GRAPH INFORMATION
-----
Number of Vertices: 5
Number of Edges : 10
Source Vertex : V0

Edges:
V0 -> V1 Weight = 6
V0 -> V2 Weight = 7
V1 -> V2 Weight = 8
V1 -> V3 Weight = 5
V1 -> V4 Weight = -4
V2 -> V3 Weight = -3
V2 -> V4 Weight = 9
V3 -> V1 Weight = -2
V4 -> V3 Weight = 7

BELLMAN-FORD RESULT
-----
Source Vertex: V0

Vertex   Shortest Distance
-----
V0       0
V1       2
V2       7
V3       4
V4      -2

FLOYD-WARSHALL RESULT
-----
      V0    V1    V2    V3    V4
-----
V0    0     2     7     4     -2
V1   -2     0     5     2     -4
V2   -7    -5     0    -3    -9
V3   -4    -2     3     0    -6
V4    2     4     9     6     0

=====
ALGORITHM COMPARISON
=====
```

Figure 2: Program execution showing graph information and shortest-path results.

11. Bellman-Ford Result

For source vertex **V0**, the shortest distances obtained are:

Vertex	Shortest Distance
V0	0
V1	2
V2	7
V3	4
V4	-2

These values represent the minimum distance from V0 to each vertex.

For example, the shortest distance from V0 to V4 is **-2**, obtained through a path containing the negative edge.

12. Floyd-Warshall Result

Floyd-Warshall produces a complete distance matrix containing the shortest distance between every pair of vertices.

FLOYD-WARSHALL RESULT					
	V0	V1	V2	V3	V4
V0	0	2	7	4	-2
V1	-2	0	5	2	-4
V2	-7	-5	0	-3	-9
V3	-4	-2	3	0	-6
V4	2	4	9	6	0

ALGORITHM COMPARISON	
Bellman-Ford	
Purpose	: Single-source shortest paths
Time Complexity	: $O(VE)$
Space Complexity	: $O(V)$
Negative Edges	: Supported
Floyd-Marshall	
Purpose	: All-pairs shortest paths
Time Complexity	: $O(V^3)$
Space Complexity	: $O(V^2)$
Negative Edges	: Supported
Analysis	

Bellman-Ford is preferred when shortest paths from one source vertex are required.	
Floyd-Marshall is preferred when shortest paths between every pair of vertices are required.	
For large graphs, Bellman-Ford generally uses less memory, while Floyd-Marshall requires a $V \times V$ matrix.	

Figure 3: All-pairs shortest-path distance matrix.

The matrix allows any source and destination pair to be queried directly after the algorithm has completed.

13. Comparison

Feature	Bellman-Ford	Floyd-Warshall
Type	Single-source	All-pairs
Output	One source to all vertices	Every pair of vertices
Technique	Edge relaxation	Dynamic programming

Negative Edges	Supported	Supported
Negative Cycle Detection	Yes	Yes
Time Complexity	$O(VE)$	$O(V^3)$
Space Complexity	$O(V)$	$O(V^2)$
Main Structure	Distance array	Distance matrix
Best Use	One source	All source-destination pairs

14. Time Complexity Analysis

Bellman-Ford

Bellman-Ford relaxes all E edges up to $V - 1$ times.

Therefore:

Time Complexity = $O(VE)$

For a sparse graph where E is close to V , this can be approximately $O(V^2)$.

Floyd-Warshall

Floyd-Warshall contains three nested loops, each iterating over the vertices.

Therefore:

Time Complexity = $O(V^3)$

As the number of vertices increases, the computational cost increases significantly.

15. Space Complexity Analysis

Bellman-Ford

Bellman-Ford stores one distance value for each vertex.

Space Complexity = $O(V)$

Floyd-Warshall

Floyd-Warshall stores a $V \times V$ distance matrix.

Space Complexity = $O(V^2)$

Therefore, Bellman-Ford is more memory-efficient for large graphs when only single-source shortest paths are required.

16. Scalability Analysis

Bellman-Ford is generally more scalable when the requirement is to calculate shortest paths from only one source. Its memory requirement remains linear with respect to the number of vertices.

Floyd-Warshall becomes expensive for very large graphs because both its running time and memory usage increase rapidly with the number of vertices.

However, Floyd-Warshall is useful for relatively smaller or dense graphs where shortest paths between all pairs are frequently required.

17. Advantages

Bellman-Ford

- Handles negative edge weights.
- Detects negative-weight cycles.
- Requires only $O(V)$ distance storage.
- Suitable for single-source shortest-path problems.

Floyd-Warshall

- Calculates all-pairs shortest paths.
 - Simple dynamic programming approach.
 - Handles negative edge weights.
 - Useful when many source-destination queries are required.
 - Provides a complete distance matrix.
-

18. Limitations

Bellman-Ford

- Slower than some shortest-path algorithms for graphs with only non-negative weights.
- Must repeatedly relax edges.
- Not ideal when all-pairs shortest paths are required.

Floyd-Warshall

- $O(V^3)$ time complexity.
 - $O(V^2)$ memory requirement.
 - Not suitable for very large graphs.
 - Maintaining a complete distance matrix requires significant memory.
-

19. Applications

Bellman-Ford

Bellman-Ford can be used in:

- Network routing
- Distributed routing systems
- Graphs containing negative weights
- Currency/arbitrage analysis
- Single-source path planning

Floyd-Warshall

Floyd-Warshall can be used in:

- Network analysis
 - Road and transportation systems
 - Social-network connectivity
 - Routing tables
 - Distance analysis between all locations
-

20. Result Analysis

The experiment demonstrates that the two algorithms address different shortest-path requirements.

Bellman-Ford calculates shortest paths from a selected source vertex and requires $O(VE)$ time and $O(V)$ space.

Floyd-Warshall calculates shortest paths between every pair of vertices and requires $O(V^3)$ time and $O(V^2)$ space.

Therefore, Bellman-Ford is preferable when only one source is important, while Floyd-Warshall is more suitable when shortest paths between all pairs are required.

21. Conclusion

The Bellman-Ford and Floyd-Warshall algorithms were successfully implemented and compared using a weighted directed graph.

Bellman-Ford is a **single-source shortest-path algorithm** with lower memory requirements, making it suitable when paths from one source are required.

Floyd-Warshall is an **all-pairs shortest-path algorithm** that provides a complete distance matrix but requires greater computational and memory resources.

The experiment shows that the choice of shortest-path algorithm depends on the graph structure, number of required paths, presence of negative weights, and available computational resources.

Thus, Bellman-Ford is generally preferable for single-source problems, whereas Floyd-Warshall is useful for all-pairs shortest-path analysis on smaller or moderately sized graphs.
